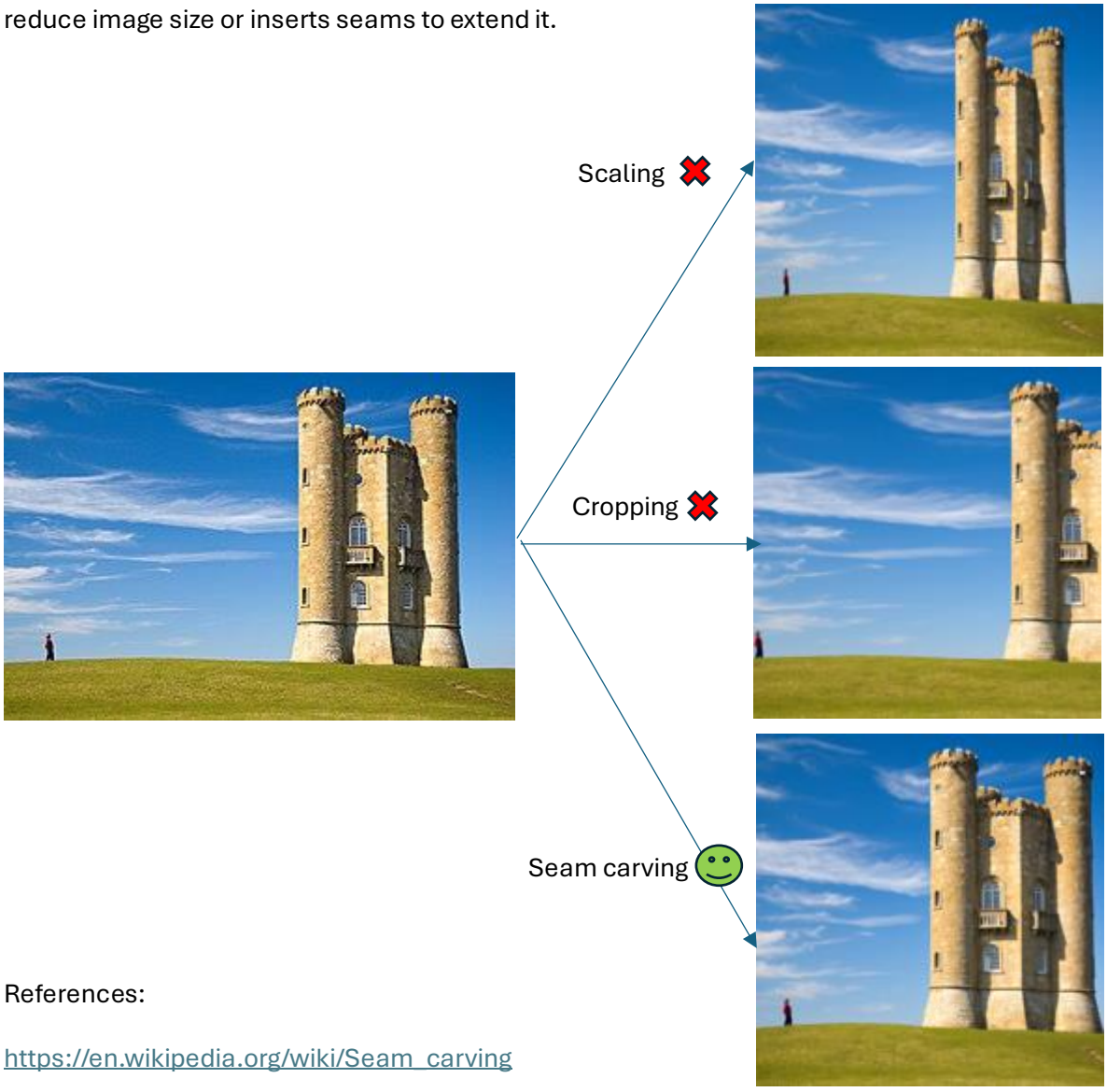
Assignment 2 (100 marks + 10 bonus)

Submission Deadline: Week 12, Fri, 20 Nov, 23:59

Image Compression by Seam Carving

Seam carving (or liquid rescaling) is an algorithm for content-aware image resizing, developed by Shai Avidan and Ariel Shamir. It functions by establishing a number of seams (paths of least importance) in an image and automatically removes seams to reduce image size or inserts seams to extend it.



References:

https://en.wikipedia.org/wiki/Seam_carving

https://dl.acm.org/doi/10.1145/1275808.1276390

https://dl.acm.org/doi/pdf/10.1145/1360612.1360615

https://avikdas.com/2019/05/14/real-world-dynamic-programming-seam-carving.html

You can choose to use any programming languages, you can use third-party libraries, including image libraries like OpenCV for image processing (reading/writing images,

computing gradient magnitude or Sobel filter) and math libraries for math computations. However, you cannot use existing functions of content-aware image resizing in the image libraries.

1. **Implement Seam carving using Dynamic programming** (for both vertical and horizontal seams). The main steps are:
   a) **Calculate the Energy Map:** For each pixel in the image, compute its energy value, which represents the disruption measure. The energy is typically calculated based on the gradient (the difference in intensity) between adjacent pixels, or by using filters like the Sobel filter to detect edges and transitions.
   b) **Use Dynamic Programming to Find the Minimum Seam:** Apply dynamic programming to find the seam (both vertically and horizontally) with the minimum cumulative energy. This seam represents the path through the image with the least disruption, as measured by the sum of the energy values along the seam.
   c) **Remove the Seam and Repeat:** After identifying the minimum energy seam, remove it from the image. Then recalculate the energy map, and repeat the process until the image is resized to the desired dimensions.
2. **Question**:
   a) Note that, this is to help you start, your final implementation does not need to follow this question.

      Given a color picture consisting of an *m x n* array, *img[1…m,1… n]*, of pixels, where each pixel specifies a triple of red, green, and blue (RGB) intensities. Specifically, we wish to remove one pixel from each of the m rows, so that the whole picture becomes one pixel narrower. To avoid disturbing visual effects, however, we require that the pixels removed in two adjacent rows be in the same or adjacent columns; the pixels removed form a "seam" from the top row to the bottom row where successive pixels in the seam are adjacent vertically or diagonally.

      i. Show that the number of such possible seams grows at least exponentially in m, assuming that $n > 1$.

      ii. Suppose now that along with each pixel *img[i, j]*, we have calculated a real valued energy or disruption measure *e[i,j]*, indicating how disruptive or energy effective it would be to remove pixel *img[i,j]*. Intuitively, the lower a pixel's disruption measure (energy), the more similar the pixel is to its neighbors. Suppose further that we define the disruption measure of a seam to be the sum of the disruption measures of its pixels. In other words, how to relate to subproblems?

   b) The gradient can be computed using various filters, such as the Sobel filter.

Convolution operations are typically used for this purpose. Discuss whether applying the Divide and Conquer method could improve the efficiency of this computation.

3. **Implement using Greedy Algorithm**. As an alternative to dynamic programming, implement the seam carving using a Greedy Algorithm. In this approach, select the seam by making a local, row-by-row decision, choosing the minimum-energy pixel at each step, without considering the entire image. Does the Greedy Algorithm provide results that are comparable to dynamic programming in terms of visual quality? Compare the seams, visual quality and running time with your Dynamic Programming implementation.
4. **Implement using Graph Cut (min-cut/max-flow)**. As another alternative to dynamic programming, reformulate seam finding as a minimum *s–t cut* problem and solve it with max-flow. This basic idea is: for a vertical seam, build a graph with one node per pixel, connect the source to every pixel in the first column and every pixel in the last column to the sink with infinite capacity, and give each edge from pixel *img[i, j]*, to its right neighbour *img [i, j+1]* a capacity equal to the cost of removing pixel *img[i, j]*, for example its energy *e[i, j]* or the forward energy $|I[i, j+1] – I[i, j−1]|$. Add infinite-capacity backward edges from [i, j+1] to [i, j], and diagonal edges from *[i, j]* to *[i−1, j−1]* and *[i+1, j−1],* so that every finite cut crosses each row exactly once and forms a connected seam. Run a max-flow algorithm (e.g. Ford–Fulkerson or Edmonds–Karp), then find the source side *S* of the minimum cut by a *BFS/DFS* from the source in the residual graph; in each row, the rightmost pixel in S is the seam pixel to remove. By the max-flow min-cut theorem the capacity of this cut equals the total cost of the seam, so the seam is optimal. Horizontal seams are handled the same way on the transposed image. Compare the seams, visual quality and running time with your Dynamic Programming implementation.
5. **Analysis and discussion**
   a) Answer the questions (Item 2).
   b) Time and space complexity.
   c) Comparison with Greedy Algorithms, Graph Cut Algorithms
   d) Any improvement, e.g., do we need to maintain the entire DP Table?, does multi-threading helps?
6. **Project Quiz: Week 13, Tue, 24 Nov, during tutorial**
   There will be a **closed-book, team-based project quiz consisting of 10–20 MCQs based on your submitted code and report**. Relevant code snippets and report extracts will be provided in the quiz. You may discuss the questions within your team, but no external materials or AI tools are allowed. The purpose is to ensure that you understand and take responsibility for the

work you submit. A moderation multiplier might be applied to your final project scores; it is tentatively as:

MCQ score	Multiplier	Interpretation
80–100%	1.00	Strong understanding
60–79%	0.95	Adequate understanding
40–59%	0.85	Significant gaps
Below 40%	0.70	Limited demonstrated understanding

7. **Bonus**: up to 10 marks will be given. The bonus will be evaluated based on the technical complexity and quality of the implementation, discussion and report:
   a) Step by step visualization, with all the seams visualized.
   b) User interaction, enable user interactively edit image sizes.
   c) CI/CD
8. **Presentation**
   a) Each person has a maximum of 2 minutes, with a total team time limit of 12 minutes.
   b) Focus on presenting the key content.
   c) Demonstrate your app and show that your program runs successfully.
   d) Record your presentation, upload it to an online platform like YouTube, and include the link in the report.
9. **Deliverables & Submission Deadline**
   a) XSite Dropbox Assignment 2 Report Only: **Report**
      i. Include your **team id** and **team members (with SIT ID)**
      ii. Answer to the Question (item 2)
      iii. Report the algorithm, design, implementation, results/evaluation as well as comparison and discussion. Highlight the algorithm and analysis.\
      iv. Include the links to:
         1. Code (github link)
         2. Presentation Video (link)
      v. Include the **AI Usage Declaration**, please specify how AI tools were utilized (e.g., during design, implementation, or report writing) and provide the main prompts used.
   b) XSite Dropbox Assignment 2 **Other Materials**
      i. Power Point Slides (if any)
      ii. Other materials like test input/outputs (if any)
   c) Submission Deadline: Week 12, Fri, 20 Nov, 23:59
   d) Project Quiz: Week 13, Tue, 24 Nov, during tutorial